Cluster Computing (2025) 28:786
https://doi.org/10.1007/s10586-025-05437-3

Performance prediction and resource adaptive adjustment for cloud-native microservices

Yu Peng[1,2] · Jia Hao[1,2] · Yang Chen[1,2]

Received: 26 December 2024 / Revised: 27 April 2025 / Accepted: 2 May 2025 / Published online: 15 September 2025
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

Abstract
Cloud-native architectures, characterized by loosely coupled microservices, enable the modular execution of specific business functions, significantly accelerating software development and reducing update complexity. However, the execution of a single application often involves hundreds of interdependent microservice nodes, where complex invocation topologies and dynamic resource demands make efficient resource allocation a critical challenge. To address this issue, we propose Convolutional Neural Network-based Prediction and Resource Adaptation Model (CPRAM), a novel framework that integrates deep learning and reinforcement learning to optimize resource allocation in cloud-native environments. The key innovations of CPRAM are as follows: (1) We propose A CNN-based performance prediction model by analyzing resource usage and response times, even with sparse or incomplete data. (2) We allocate CPU and memory via a Deep Deterministic Policy Gradient (DDPG) based mechanism so as to maximize global resource utilization. We evaluate CPRAM using DeathStarBench, a widely adopted cloud-native microservices benchmark, and deploy our experiments on Docker Swarm. Extensive experimental results demonstrate that CPRAM outperforms state-of-the-art baselines, achieving significant resource savings of at least 22.8% in CPU and 21.91% in memory utilization while consistently meeting end-to-end QoS targets.

Keywords Cloud native · Microservices · Performance prediction · Resource allocation · Convolutional Neural Network · Deep Deterministic Policy Gradient

1 Introduction

Cloud-native architecture has emerged as a cutting-edge paradigm for developing highly automated, scalable, and resilient applications. Unlike traditional monolithic cloud systems, cloud-native architectures decompose complex applications into modular, loosely coupled microservices that communicate through lightweight protocols such as message queues or remote invocations [1]. This architectural shift enables developers to deploy and update individual functional modules independently, significantly accelerating the software development lifecycle [2]. However, the fine-grained nature of microservices introduces new challenges in ensuring end-to-end quality of service (QoS). A single user request, such as an e-commerce transaction or a streaming media service, often involves hundreds of interdependent microservices, making it difficult to monitor and optimize the performance of critical-path microservices in real time [3, 4].

Existing research has made significant strides in addressing resource management challenges in cloud-native environments. For instance, Bhardwaj et al. [5] proposed Cilantro, a performance-aware resource allocation framework that employs online learning to dynamically adjust resources based on real-time performance feedback. Similarly, Qiu et al. [6] introduced FIRM, a machine learning-based framework that predicts microservice performance and adapts resources to prevent service-level objective (SLO) violations. Guo et al. [7] developed PARA, which optimizes resource allocation by considering

✉ Jia Hao
haojiaynu@qq.com

[1] Key Laboratory of Education Informatization for Nationalities, Ministry of Education, Yunnan Normal University, Kunming 650500, China

[2] Yunnan Key Laboratory of Smart Education, Yunnan Normal University, Kunming 650500, China

dynamic resource demands and employs heuristic algorithms to maximize long-term performance. Despite these advancements, several limitations remain unresolved.

First, existing approaches often require extensive manual preprocessing of resource-related features before they can be fed into machine learning models, increasing the computational and memory overhead [8]. This is particularly problematic for sparse features, such as CPU idle times, which are frequently zero in profiling data but require significant preprocessing effort. Second, most resource management frameworks focus on individual microservices, neglecting the global invocation relationships among interdependent microservices. This oversight can lead to suboptimal resource allocation and degraded end-to-end QoS [9].

To address these challenges, we propose Convolutional Neural Network-based Prediction and Resource Adaptation Model (CPRAM), a novel resource management framework that leverages deep learning and reinforcement learning to optimize resource allocation in cloud-native environments. CPRAM uses the end-to-end response time as the primary QoS indicator and employs a two-stage approach: (1) a Convolutional Neural Network (CNN)-based performance prediction model that accurately forecasts microservice performance even with sparse or incomplete data, and (2) a Deep Deterministic Policy Gradient (DDPG)-based resource allocation mechanism that dynamically adjusts CPU, memory, and I/O resources to meet QoS requirements while maximizing global resource utilization [10].

The key contributions of this paper are as follows:

- **CPRAM Framework**: We propose a novel resource management model that integrates CNN-based performance prediction with DDPG-based resource allocation, enabling accurate performance forecasting and dynamic resource adaptation in cloud-native environments.
- **Global Optimization**: Unlike existing approaches that focus on individual microservices, CPRAM considers the global invocation relationships among microservices, ensuring optimal resource allocation across the entire system.
- **Experimental Validation**: We evaluate CPRAM using DeathStarBench [11], a widely adopted cloud-native microservices benchmark, and deploy our experiments on Docker Swarm. Extensive experimental results demonstrate the effectiveness and efficiency of CPRAM in meeting QoS requirements and optimizing resource utilization.

Research Motivation: The motivation behind this work stems from the growing complexity of cloud-native applications and the limitations of existing resource management frameworks. While current solutions have made progress, they often fail to address the challenges posed by sparse data and global microservice interdependencies. CPRAM aims to bridge this gap by providing a robust, automated solution for performance prediction and resource allocation.

Paper Organization: The remainder of this paper is organized as follows. Section 2 reviews related work in resource management for cloud-native microservices. Section 3 presents the design and implementation of CPRAM, including the CNN-based prediction model and the DDPG-based resource allocation mechanism. Section 4 describes the experimental setup and evaluates the performance of CPRAM. Finally, Sect. 5 concludes the paper and discusses future research directions.

2 Related work

2.1 The prediction of cloud-native microservice

Signiant efforts have been dedicated to the performance prediction of microservices to ensure the effectiveness of follow-up resource allocation.

For example, Rzadca et al. [12] proposed Autopilot, a vertical and horizontal algorithm scaler utilized by Google for predicting resources. It leverages the optimized sliding window algorithm and a reinforcement learning-based method to enhance system utilization. Guo et al. [7] proposed the PARA strategy, which took the service degradability, dynamic resource demand, the characteristics of MEC and cloud resource supply into consideration. They analyzed the historical resource demand, and then established an enhanced-heuristic algorithm based resource allocation model aiming to maximize long-term performance. Zhang et al. [13] proposed Sinan, a scalable and QoS-aware interactive microservice resource manager. They first used a set of scalable and verified machine learning models to determine the impacts from dependencies among microservices on performance, and then predict the QoS under the current resource allocation. Then, resources were allocated to each layer in a manner that preserved the end-to-end application delay. Daniel et al. [14] introduced the Clipper system, which implemented a dual-layer architecture consisting of a model abstraction layer and a model selection layer, enabling unified management and optimization of multiple machine learning models to support online model selection and real-time performance prediction. Christina et al. [15] proposed a nature-inspired microservice resource management and dynamic scheduling framework in cloud centers. In this study, a predictive scheduling strategy for containerized microservices was developed using the multiverse

optimization algorithm, which effectively reduced the average response time and improved the efficiency of resource utilization of the whole system. Roland et al. [16] used regression model to estimate resource usage and performance of data-intensive applications based on microservices, and then minimized CPU and memory reservation under QoS constraints with the formed linear programming. Michel et al. [17] proposed a discrete event simulator for simulating and predicting the performance of cloud-native microservice chains. Jeong et al. [18] proposed the HiPerRM system, which utilized the SCINet model to predict future workload and optimized container resource management with reverse case normalization technology. Sotirios et al. [19] proposed the Rusty system, which utilized the LSTM to predict resource and energy consumption with fine-grained performance counters, and demonstrated high prediction accuracy under various interference scenarios, enabling the management of runtime interference in multi-tenant systems.

Current methods for microservice performance prediction often rely on traditional machine learning techniques or manual feature engineering, which suffer from several limitations. Firstly, these approaches typically require extensive preprocessing of resource usage data, such as handling sparse features (e.g., CPU idle times that are frequently zero) or normalizing data, which increases computational overhead and complexity. Secondly, they often fail to capture the intricate relationships and dependencies among microservices, as they treat each node in isolation rather than considering the global invocation topology. In contrast, Convolutional Neural Networks (CNNs) offer significant advantages for microservice performance prediction. CNNs can automatically extract meaningful patterns from raw data, eliminating the need for manual feature engineering. Their ability to process spatial and structural information, such as the invocation relationships among microservices, enables them to model complex dependencies more effectively. Additionally, CNNs are robust to sparse or incomplete data, making them well-suited for the dynamic and heterogeneous nature of cloud-native environments. These capabilities allow CNNs to deliver more accurate and reliable performance predictions, even in scenarios where traditional methods struggle.

2.2 The resource adjustment for cloud-native microservice

The resource adjustment for the microservice node mainly involves in monitoring, analyzing, and then adjusting resources such as CPU, memory, and I/O network based on the real-time resource requirements, so as to optimize the task performance, reduce the cost of cloud servers, and then maintain the high availability and reliability.

For example, Zhang et al. [20] proposed a lightweight resource management system, Ursa, which used an analytical model to decompose the end-to-end service level agreement (SLA) into each microservices, and then mapped these SLAs to the separate resource allocations of each microservice layer, so as to halt the corresponding execution when latency exceeded the layer-wise SLA promptly. Wang et al. [21] introduced the Autothrottle framework, which optimized microservice resource allocation for service level objectives (SLO) through a two-tier management structure. They utilized application-level learning controllers to set performance targets, and then employed service-level heuristic controllers to adjust CPU allocation dynamically. Luo et al. [22] proposed the Erms system, which utilized the piecewise linear feature of microservice delay, combined with global optimization and priority scheduling policies, and then optimized resource sharing and scheduling among microservices to reduce the probability of SLA violation, so as to improve resource utilization and performance significantly. Chen et al. [23] proposed a cloud running system, PARTIES. Through online monitoring and a resource partitioning mechanism, PARTIES adjusted resource allocation to meet the QoS requirements of co-scheduled workloads and maximize the throughput of servers dynamically. Guo et al. [24] proposed the POBO framework, which combined pessimistic and optimistic Bayesian optimization. They demonstrated the efficiency and accuracy of real microservice applications through primal duality and penalty-based optimization methods, while ensuring the maximization of SLA and resource efficiency. Ahyoung et al. [25] utilized genetic algorithms to simulate different resource allocation schemes, and then optimized the resource adjustment for container deployment in heterogeneous cloud environments. Ataollah et al. [26] proposed SHOWAR, which optimized resource allocation and microservice scheduling through real-time monitoring of resource usage and performance indicators. It combined vertical and horizontal algorithm scaling technology, leading to significant improvements in resource utilization and system performance. Piotr et al. [27] used machine learning methods to deal with the problem of overestimating resource utilization of services using cloud computing, and gained great practicability when planning cloud resource reservation for network services. Reena et al. [28] combined the concepts of autonomic computing, linear regression and Bayesian learning to propose a framework for providing resources in an optimal way, using existing prior information to predict resources. Antha et al. [29] proposed an adaptive scheduling algorithm based on dynamic pricing of combinatorial auction allocation mechanism to improve resource utilization and user satisfaction through dynamic pricing of combinatorial auction.

Current methods for microservice resource allocation often rely on heuristic rules or static optimization techniques, which exhibit several critical limitations. Firstly, these approaches typically lack adaptability to dynamic resource demands, as they are designed based on fixed thresholds or historical data, making them ineffective in handling the highly variable workloads of cloud-native environments. Secondly, they often focus on optimizing individual microservices in isolation, neglecting the global interdependencies and invocation relationships among microservices, which can lead to suboptimal resource utilization and degraded end-to-end performance. In contrast, the Deep Deterministic Policy Gradient (DDPG) algorithm offers significant advantages for microservice resource allocation. As a model-free reinforcement learning method, DDPG can dynamically adapt to changing workloads and resource requirements in real time, ensuring optimal resource allocation under varying conditions. Moreover, DDPG is capable of learning global optimization policies by considering the interactions and dependencies among microservices, enabling it to maximize system-wide resource efficiency while meeting quality of service (QoS) targets. Its ability to handle continuous action spaces also makes it particularly suitable for fine-grained resource adjustments, such as allocating CPU cores, memory, or I/O bandwidth. These capabilities position DDPG as a powerful solution for addressing the complexities of resource management in cloud-native systems [33–36].

3 The construction of CPRAM

In this chapter, we give the details of CPRAM-based microservice performance prediction and resource adaptive adjustment in a cloud-native environment. We first introduce the overall architecture of CPRAM, and then give the data preparation process. Then, the CNNRPM-based resource prediction and DDPG-based resource allocation will be introduced respectively.

3.1 The brief introduction of CPRAM

CPRAM, whose overall architecture is given in Fig. 1, comprises three components, namely (1) the tracking of cloud-native microservices, (2) performance prediction via CNN, and (3) DDPG-based resource allocation.

(1) Track of cloud-native microservices. We deploy the DeathStarBench test suite via Docker, a container management tool. Then, we use the Linux resource tracking command *vmstat* to collect the resource usage of microservices at a given time interval. At the same time, the Linux resource isolation mechanism *Cgroup* is used to implement resource restrictions on each microservice. In this way, we can track the resource usage of every microservice nodes in the non-interference state, and whose performance (i.e., response time) is then collected by the distributed tracking system *Jaeger*.

Next, the benchmark of Phoronix-Test-Suite is used to simulate the interference from the perspective of CPU, memory, I/O, and we use the above-mentioned method to collect the resource-performance data correspondingly. Finally, the feature engineering method is adopted to conduct data pre-processing, so as to complete the performance tracking of microservice nodes under the cloud-native environment.

(2) The construction and prediction via CNN. The collected resource-performance tabular data is converted into grayscale images, which are the input of a CNN, and the key resource-related features then can be extracted through the operations of convolution and pooling, thus completing the training. Based on CNN, we predict the performance of microservice when given the resources.

(3) DDPG-based resource allocation. According to the prediction of CNNRPM, after obtaining the call dependencies among microservices, each of them is regarded as an agent. Then, we adopt the Actor-Critic algorithm with the attention mechanism to train the DDPG in a decentralized way, and learn the best resource allocation strategies under different workloads to ensure the QoS of end-to-end tasks at the same time.

3.2 Data-preparation: track of cloud-native microservices

Firstly, we need to prepare the model training data from all of the microservice nodes, including the usage of CPU, memory and other resources under the critical path, as well as the corresponding response time. Then, in order to make the trained CPRAM model more universal, we injected artificial interference into each microservice node to simulate the resource competition among different resources.

The dataset is collected from the container engine Docker and the benchmark DeathStarBench, which is an open-source toolset for cloud-native environment that builds six interactive and delay-sensitive services (such as social networking, video streaming, e-commerce, and cluster control services) based on microservice architectures, and covers the current popular cloud and edge applications. The data is collected in the following way. First, we installed Docker on two physical hosts, and then deploy DeathStarBench, then the Linux profiling tool *vmstat*[1] is used to collect the resource usage of microservices from each application at a given interval, including the

[1] Vmstat: https://www.man7.org/linux/man-pages/man8/vmstat.8.html,2024/5/7.

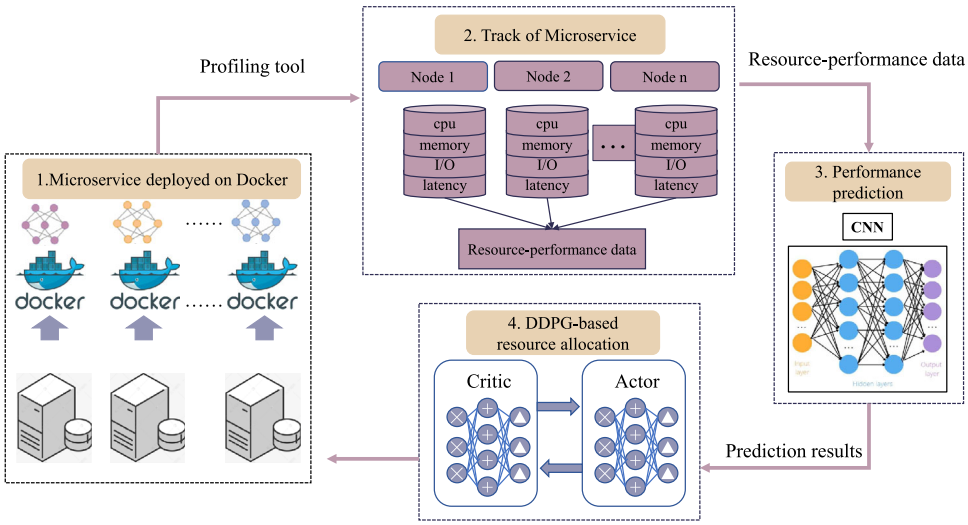


Fig. 1 Model architecture

Table 1 Resource characteristics and performance collection

Resource-related features					
Collect by *vmstat*		Description	Initialized by *Cgroups*		Description
Process level	procs.r/b	Number of running and waiting processes	CPU	cpu.shares cpu.cfs_period_us cpuset.cpus	CPU allocation for microservices
CPU level	cpu.us/sy/id/wa/st	CPU percentage for user, system, idle and I/O wait	Memory	memory.limit_in_bytes memory.swappiness	Memory allocation for microservices
Memory level	memory.swpd/free memory.cache/buff swap.si/so	Allocated and swapped memory capacity			
I/O block level	bi/bo	Read and written data from block device			
System level	system.in/cs	Number of interrupts and context switches per second			

CPU and memory usage, and I/O read/write rate. Next, *Cgroups*,[2] the physical resource isolation mechanism provided by the Linux kernel, is used to adjust the resource allocation of each node, and *Jaeger*,[3] a distributed tracking system, is used to collect the response time of microservice nodes and the call topology diagram between microservices, so as to complete the collection of resource-performance data under the non-interference state. The details of data features are shown in Table 1.

Then, we inject different types of interference on each node to simulate the actual competitive scenarios. This injected program is implemented through the benchmark of Phoronix-Test-Suite,[4] whose details are given as follows:

- CPU-intensive Interference. We simulate the competition of CPU by using *smallpt* from the Phoronix-Test-Suite. The *smallpt* simulates the ray tracing by emitting light and tracing its path continuously, as well as interacting with objects in the scene. This process requires a large amount of computing resources, so it can be used to achieve the preemption of CPU resources.
- I/O-intensive Interference. We use *Tiobench* from the Phoronix-Test-Suite to implement I/O interference.

[2] Cgroup: https://www.kernel.org/doc/html/latest/admin-guide/cgroup-v2.html,2024/5/7.

[3] Jaeger: https://www.jaegertracing.io/,2024/5/7.

[4] *Phoronix-Test-Suite*: https://www.phoronix-test-suite.com/,2024/5/7.

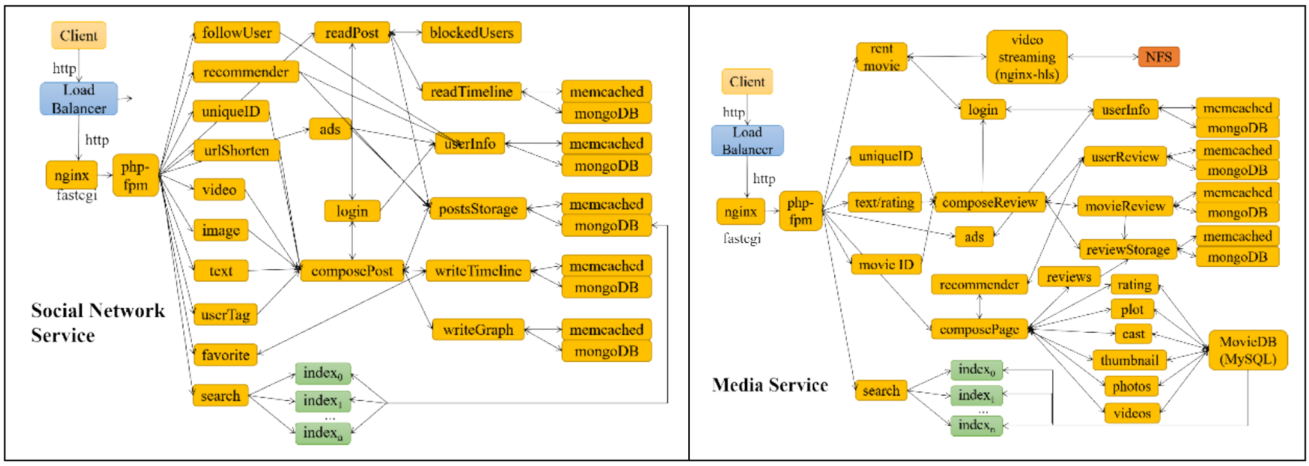


Fig. 2 The topologies of social network and media service

Tiobench can simulate a series of file system operations, including read, write, and random access of the I/O disks.

- Memory-intensive Interference. We write a script in Linux system to achieve the preemption of memory resources. Firstly, we create a temporary directory, which is mounted as a temporary memory-based file system *tmpfs*. Then, we specify the size of *tmpfs*, and then write data to the block file under the directory, thus consuming the system memory resources.

Next, we delete the outliers and the duplicate data, and then smooth the noise from the collected dataset and fill in the missing values, so as to complete the data preparation.

For example, the call topologies of social network and media service from DeathStarBench are illustrated in Fig. 2

Then, we the details for microservice tracking.

In the execution history graph G, the critical path (CP) is defined as the path with the longest duration from the beginning to the end of the user request, and the change of the end-to-end delay of an application is usually caused by insufficient or excessive resources of one or more microservices on its CP. Therefore, we need to first identify the CP in the execution history graph. Then, resource prediction is carried out on the microservices on it, so as to accurately find out the microservices with resource allocation problems and make corresponding resource adjustment. Algorithm 1 is a CP extraction algorithm.

The critical path extraction algorithm is crafted to systematically identify the longest path (critical path) from a microservice execution history graph. The process begins by initiating from the current node, initiating the path, and including the current node within it. If the current node lacks child nodes, the path terminates, and the current path is returned. Conversely, the algorithm retrieves the last returned child node and proceeds with a recursive search for the longest path from that particular child. It iterates through all the child nodes of the current node, and if each child appears prior to the last returned child, the recursive search for the longest path continues from that child, ultimately returning the constructed longest path. By scrutinizing the performance of nodes along the microservice critical path (CP), we can identify the nodes causing end-to-end violations and accordingly adjust their resource allocation to guarantee end-to-end QoS.

3.3 Performance prediction: the construction of CNN-based model

We first give the definitions of the symbols involved in this paper.

Let $V = \{V_1, V_2, ..., V_N\}$ denote the set of N features collected by *vmstat*, where $V_i(i \in 1,2,...,N)$ denotes the ith feature, including the number of running processes, memory usage, and CPU usage, etc., and V_{ij} is the jth value of the ith feature $(i \in 1,2,...,N)$.

Let $C = \{C_1, C_2, ..., C_M\}$ denote the set of *cgroup* allocated features, where C_{ij} is the jth value of the ith$(i \in 1,2,....,M)$ feature.

Let T be each microservice node performance metric collected by *jaeger*, i.e., the response time of a microservice node. $T_i(i \in 1,2,...,t)$ is the ith value of T.

Based on the features V, C and the response time T, we construct the CNN-based performance prediction model, which is divided into two steps, i.e., transforming the grayscale image based on the resource-performance data, and then constructing the CNN model to predict the response time of the microservices nodes under the specific resource allocation. The details are shown in Fig. 3.

First, we transform the grayscale image based on the resource-performance data. Specifically, for the resource-

Algorithm 1 Critical path (CP) extraction

Input: G: Microservice execution history graph G;
cn: currentNote
lrc: currentNote.lastReturnChild
cnc:currentNote. childNotes
sn:startNode
Output: CP # Critical Path
function LongestPath(G,cn)
  CP ← ∅ # Initializes an empty path set to store the longest path from CP.
  CP.add(cn) # Add currentNode to the path.
  if cnc == None **then**
    Return CP
  else
    lrc ← $currentNote.lastReturnChild$ # Get the last returned child of the current node
    CP.extend(**LongestPath**(G,lrc)) # Recursively extend the path with the longest path from lrc
    for each cn in cnc **do** # Iterate over all child nodes cn of the current $currentNode$
      if cn.happensBefore(lrc) **then** # Check if the child node occurs before the last returned child
        CP.extend(**LongestPath**(G,cn)) # Starting with cn, call LongestPath recursively.
      end if
    end for
  Return CP # Return critical path
function FindCP(G):
    sn ← G.getStartNode() // Get the start node of the graph
    return **LongestPath**(G, sn) // Find and return the longest path starting from the start node

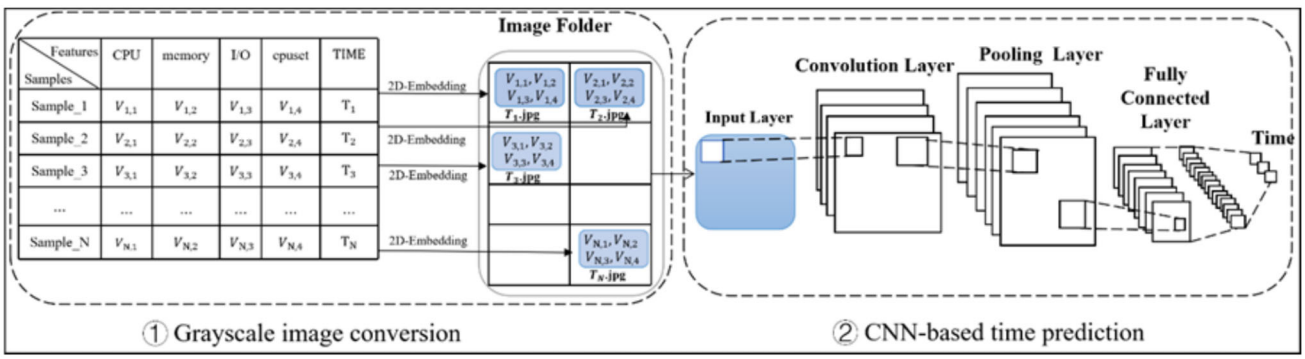


Fig. 3 CNNRPM model diagram

performance samples, each of which has multiple feature columns and a label column, where the feature columns are the set of relevant resource features V captured by *vmstat* and the set of resource allocation features C captured by *cgroup*, and the label column is the corresponding performance metric T. Then, we normalize each resource feature from each sample into the range of [0,1], which will be then mapped with the grayscale values in a predefined character set (where each character corresponds to a grayscale level). The processed characters are then drawn in a blank image using image processing libraries in a top-to-bottom, left-to-right order or some specific layout strategy to ensure each feature is correctly positioned into a two-dimensional matrix, and after performing the necessary filling and alignment operations, a gray-scale image feature image is formed for each sample.

For example, the step (1) in Fig. 3 illustrated the process of gray-scale image transformation. Assume that there are N samples and each of them has four features (i.e., number of CPU, memory capacity, and I/O throughput) and a label column T_i, then after 2D embedding of sample 1, we can generate a grayscale image named T_1.jpg, i.e., the image with performance T_1 contains four features at different locations.

The second step involves constructing the CNN and subsequently forecasting the performance of microservice nodes under a specified resource allocation. The data input layer of CNN primarily pre-processes raw image data through techniques such as mean removal, normalization, and PCA/SVD dimensionality reduction. The convolution layer executes convolution operations on the input data, employing customized convolution kernels to extract image features. The activation layer employs an activation function to introduce nonlinear mapping, while the pooling layer reduces feature dimensions, often placed after the convolutional layer to yield smaller, refined features. We iterate the convolution, activation, and pooling operations until convergence, and the resulting well-trained CNN is then utilized for accurate prediction of microservice node performance.

In this paper, the CNN is trained via back propagation algorithm to optimize its parameters and the loss function is shown in Eq. (1).

$$L = -\frac{1}{n} T_i \ln a_i, \tag{1}$$

where x denotes the sample, T denotes the collected time, a denotes the predicted output, and n denotes the total number of samples. The loss function measures the similarity between the actual output probability and the expected ones, and the smaller the value of cross entropy, the closer the two probability distributions are.

The specific algorithm of CNN-based performance prediction is shown in Algorithm 2.

For the time complexity of Algorithm 2, we analyze step by step. For the FineTuneCNN(), it initializes and then fine-tunes the CNN model. Assuming the time complexity of FineTuneCNN() is $O(C)$, where C represents the complexity of the CNN model, which typically depends on the number of layers, the number of parameters, and the size of the training data. The outer loop iterates over the dataset D

Algorithm 2 CNNRPM construction

Input: Resource-performance data set $D = \{feature,\ label\}$ # n pieces of data

FineTuneCNN()

Output: The Prediction of response time T

1: F_{cnnrpm} = FineTuneCNN() # CNNRPM model initialization

2: **for** D_i **in** D **do:**

3: $image$ = [] # Initialize an empty $image$

4: **for** $feature$ **in** D_i **do:**

5: image[$feature$.position] = $feature$.value # Iterate over each cell of the table data, filling its value into the corresponding image position.

6: **end for**

7: $T = F_{cnnrpm}$ (image[$feature$.position]) # The CNNRPM model was used to predict the response time T

8: **end for**

9: **return** T

with n data points, so the time complexity of the outer loop is $O(D)$. The inner loop, iterating over m features of each data point D_i, has the time complexity of $O(m)$. For each data point, the F_{cnnrpm} function is called to make a prediction. Assuming the time complexity of the forward propagation of the CNN model is $O(F)$, where F represents the computational complexity of the CNN model, which typically depends on the input data dimensions, kernel size, number of layers, etc. Thus, combining the time complexity of the above steps, the total time complexity can be expressed as $O(C) + O(n) * (O(m) + O(F))$.

3.4 Resource allocation: construction of DDPG-based model

This section first defines the resource allocation problem for microservice nodes. Then, a method based on deep reinforcement learning (DDPG) is adopted to coordinate the allocation of each type of resources, aiming to meet the end-to-end QoS of the task, while improving the resources utilization of the whole system as much as possible.

3.4.1 Problem definition in resource allocation model based on DDPG

In this study, the resource allocation problem based on deep reinforcement learning is first modeled as a Markov chain. In a cloud-native architecture, an application typically consists of multiple microservice nodes, each of which running a specific function. Assuming that an application consisting of n microservice nodes, which can be represented as $M = \{M_1, M_2, M_n\}$ where $M_i (i \in [1, n])$ represents the i-th one. The Markov model can be expressed as the tuple$\partial = \{S, A, R, P\}$, where S represents the state space, A represents the action space, R represents the reward set, and P represents the state transition probability. The current state $s_{i,t}$ is a specific value of state S for nodeM_i. At each t moment, each microservice node M_i will take corresponding actions $a_{i,t}$ based on specific policies according to its current state$s_{i,t}$, and then produce the next state $s_{i,t+1}$ through the state transition probability function$P\left(s_{i,t+1}|s_{i,t}, a_{i,t}, , a_{n,t}\right) : S \times A \rightarrow [0,1]$, thus M_i receives the instant rewards $r_{i,t}$ through status and action functions. The basic components of $\partial = \{S, A, R, P\}$ are described in detail below.

- $S = \{s_1, s_2, \ldots, s_i, s_{i+1}, \ldots, s_n\}$ indicates the state space, $s_{i,t} = \{c_{i,t}^{cpu}, c_{i,t}^{memory}, c_{i,t}^{io}\}$ indicates the resources allocated for the i-th node at time t, where $c_{i,t}^{cpu}$ is the number of cores used by the CPU, and $c_{i,t}^{memory}$ is the memory capacity, and $c_{i,t}^{io}$ is the I/O throughput.
- $A = \{a_1, a_2, \ldots, a_i, a_{i+1}, \ldots, a_n\}$ indicates the action space, which is the set of executable actions, where $a_i = \{x_i^{cpu}, x_i^{memory}, x_i^{io}\}$ indicates the i-th resource allocation scheme on the microservice node.
- The reward is the feedback obtained by the environment based on the action taken by the current microservice node M_i, which represents the value obtained after microservice node M_i taking action $a_{i,t}$ in the current state $s_{i,t}$. There are two kinds of rewards: positive feedback and negative feedback. When the scheduled resources can meet the needs of microservice nodes and are lower than a specific threshold, the reward is positive. Otherwise, the reward is negative. The goal of microservice node is to maximize the reward function given in Eq. (2):

$$R_i = \sum_{j=0}^{\infty} \gamma^j r_{i,t+j} \tag{2}$$

where γ ($\gamma \epsilon [0,1]$) is the discount factor, which measures the extent to which future rewards affect current actions. $r\left(x_{i,t}\right)$ is the immediate reward obtained by the microservice node when it takes action $a_{i,t}$, and it can represent the ratio of the actual resource usage, whose calculation is given in Eq. (3).

$$r\left(x_{i,t}\right) = \alpha \frac{c_{i,t}^{cpu}}{x_{i,t}^{cpu}} + \beta \frac{c_{i,t}^{memory}}{x_{i,t}^{memory}} + \gamma \frac{c_{i,t}^{io}}{x_{i,t}^{io}} \tag{3}$$

where α, β and γ represent the weight, and $x_{i,t}$ represents the set of resource quotas assigned to microservice node M_i at time t, namely $\{x_{i,t}^{cpu}, x_{i,t}^{memory}, x_{i,t}^{io}\}$. For example, a larger $c_{i,t}^{cpu}/x_i^{cpu}$ ratio indicates that the CPU resource allocation share allocated to the microservice node is closer to the actual demand. The reward function defines the value that if the resource allocation is proper.

- The status value s_t, action value a_t, reward value r_t of each training and the status value s_{t+1} of the next moment are stored as a tuple:

$$e_t = (s_t, a_t, r_t, s_{t+1}) \tag{4}$$

All training samples are stored in the experience buffer D, namely $D = \{e_1, e_2 \ldots, e_t\}$.

3.4.2 Construction of resource allocation model based on DDPG

The resource allocation model based on DDPG proposed in this paper is shown in the Fig. 4. Firstly, the Actor network captures the observation information through the agent and then uses deterministic strategy to get the corresponding action. Second, the Critic network captures the action and state information of all microservice nodes and evaluates the output action of the Actor network.

In this paper, we adopt an architecture of dual neural network to realize the DDPG algorithm, which contains the

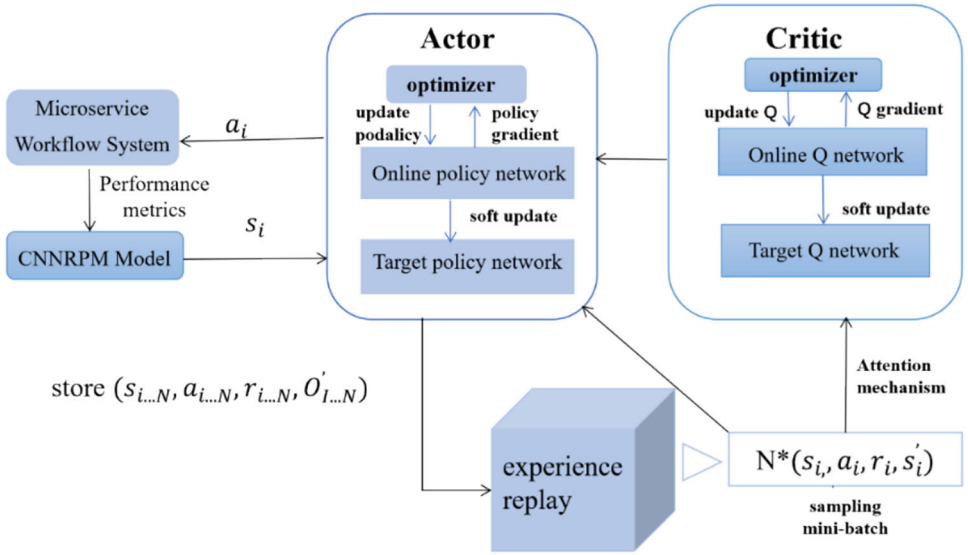


Fig. 4 Resource allocation model diagram based on DDPG

policy function in Actor network and the value function in Critic network, namely Online network and Target network. The Critic network uses the value function $Q^{\pi}(s,a)$ to judge the performance of Actor, and then guide whose actions in the next stage. The Actor uses the policy function π to update the value method of the strategy π according to the optimization result of the value function by the Critic. In Actor networks, the policy function $\pi_\theta : S \rightarrow A$ is a mapping from the state space to the action space, indicating that the agent selects action a_t in the state s_t, executes the action and moves to the next state s_{t+1} with probability $f(s_t, a_t)$, while receiving the reward r_t from the environment feedback, where θ is the target network parameter of the Actor.

Generally speaking, DDPG consists of four neural networks: Target network in Actor network, Online network, Target network in Critic network and Online network. According to the current state value, the Online network in the Actor network provides the policy function of the current state, the policy function $\pi_\theta(s_i) : S \rightarrow A$ is a mapping from state space to action space, indicating that the agent selects action a_t in state s_t, executes the action and moves to the next state s_{t+1} with probability $f(s_t, a_t)$, while receiving reward r_t from environment feedback.

The Online network in the Critic network provides the Q value function $Q_\omega(s_t, a_t)$ according to the current state—action output in the Actor network, which is used to evaluate the current policy. The Q value function is defined as follows.

$$Q_\omega(s_t, a_t) = E_{r_{i\geq t}, s_{i>t} \sim E, a_{i>t} \sim \pi}[R_t | s_t, a_t] \tag{5}$$

If the Target policy is deterministic, we can describe it as the function $\mu : S \leftarrow A$, in order to increase the agent's exploration in the environment, DDPG adds a Gaussian noise function to the behavior policy, and the Target network of the Critic network is used to estimate the Q-value function of the state-action at the next moment:

$$Q^{\mu}(s_t, a_t) = E_{r_t, s_{t+1} \sim E}[r(s_t, a_t) + \gamma Q^{\mu}(s_{t+1}, a_{t+1})] \tag{6}$$

where $a_t = \pi_\theta(s_i) + \varepsilon$, ε represents the exploration noise on the behavioral strategy.

The network parameters are updated by minimizing the mean square error between the current Q value and the target Q value, with θ^Q as the update parameter. The error function is:

$$L(\theta^Q) = E_{s_t \sim \rho^{\beta}, a_t \sim \beta, r_t \sim E}\left[\left(Q((s_t, a_t|\theta^Q) - r(s_t, a_t) + \gamma Q^{\mu}(s_{t+1}, \mu(s_{t+1})|\theta^Q))^2\right)\right] \tag{7}$$

Then, by maintaining a parameterized Actor function $\mu(s|\theta^{\mu})$, this function guides the current policy by deterministically mapping the state to a particular action. The Actor update formula is as follows:

$$\begin{aligned} \nabla_{\theta^{\mu}} J &\approx E_{s_t \sim \rho^{\beta}}\left[\nabla_{\theta^{\mu}} Q(s, a|\theta^Q)|_{s=s_t, a=\mu(s_t|\theta^{\mu})}\right] \\ &= E_{s_t \sim \rho^{\beta}}\left[\nabla_{\theta^{\mu}} Q(s, a|\theta^Q)|_{s=s_t, a=\mu(s_t)} \nabla_{\theta_{\mu}} \mu(s|\theta^{\mu})|_{s=s_t}\right] \end{aligned} \tag{8}$$

where J represents the state value function, the value of which represents the cumulative expectation in each state adjustment process.

The target network parameters ω' and θ' are updated through soft updating to ensure the parameters can be updated slowly, thus improving the stability of learning:

$$\omega' \leftarrow \varepsilon\omega + (1-\varepsilon)\omega' \tag{9}$$

$$\theta' \leftarrow \varepsilon\theta + (1-\varepsilon)\theta\prime \tag{10}$$

Based on the above process, when receiving feedback that the end-to-end response time of the CNNRPM model exceeds a specific threshold range, we first initialize the experience buffer D, the Actor network, and the Critic network.

Initializing the state at the beginning of each episodes, the Actor network performs a specific action a_i={x_i^{cpu}, x_i^{memory}, x_i^{io}} for increasing the amount of resources such as

CPU, memory and I/O allocated, based on the state of the microservices node when the current actual amount of CPU, memory and I/O resources used $s_i = \{c_i^{cpu}, c_i^{memory}, c_i^{io}\}$. And the agent gets the corresponding reward $r(x_i) = \alpha \frac{c_i^{cpu}}{x_i^{cpu}} + \beta \frac{c_i^{memory}}{x_i^{memory}} + \gamma \frac{c_i^{io}}{x_i^{io}}$ and the next state $s_{i,t+1} = \left\{c_{i,t+1}^{cpu}, c_{i,t+1}^{memory}, c_{i,t+1}^{io}\right\}$. The status value s_t, action value a_t, reward value r_t of each training and the status value s_{t+1} of the next moment are stored as a tuple $e_t = (s_t, a_t, r_t, s_{t+1})$, which will be put into the experience buffer D. The above operations are repeated until the experience buffer D reaches the minimum sampling criteria, and then N samples are randomly selected from D to train the Actor network to update the parameters θ of the policy function π and the Critic network. The specific resource allocation process based on DDPG is shown in Algorithm 3.

In the implementation of Algorithm 3, the resource allocation model leveraging DDPG first establishes an Actor network, which determines an action A based on the current environment state S. Subsequently, the network receives the corresponding reward R and observes the next state $S\prime$. Next, a Critic network is formulated to evaluate the Actor's strategy and steer actions for the subsequent phase. The deterministic policy is employed to update the Critic network, aiming to maximize the Q value and reward. Moreover, during the training of the Q network, experience replay is utilized to break the correlation between data samples, ensuring they adhere to an independent and identical distribution. This approach reduces the variance in parameter updates, enhances convergence speed, and ultimately ensures a reasonable training duration.

Lastly, we give an example including performance prediction and resource allocation to better clarify our idea (Fig. 5).

Algorithm 3 DDPG-based adaptive resource allocation

Input:	Initialize the experience buffer D, the Actor network, and the Critic network
Output:	Resource allocation action
1:	$\theta^{Q'} \leftarrow \theta^{Q}, \theta^{\mu'} \leftarrow \theta^{\mu}$ # Initialize the target network weight parameters Q' and μ' Initialize the experience buffer D
2:	**for** each episode=1 to M **do**
3:	Random noise ε initialization
4:	Get the initial state value $s_i = \{c_i^{cpu}, c_i^{memory}, c_i^{io}\}$
5:	**for** t = 1 to T **do**
6:	a_i=$\{x_i^{cpu}, x_i^{memory}, x_i^{io}\}$ # Perform action a_i
7:	$r(x_i) = \alpha \frac{c_i^{cpu}}{x_i^{cpu}} + \beta \frac{c_i^{memory}}{x_i^{memory}} + \gamma \frac{c_i^{io}}{x_i^{io}}$ # Get the rewards $r(x_i)$
8:	$s_{i,t+1} = \{c_{i,t+1}^{cpu}, c_{i,t+1}^{memory}, c_{i,t+1}^{io}\}$ # Gets the next state value $s_{i,t+1}$
9:	Save the tuple $e = (s_t, a_t, r_t, s_{t+1})$ to experience buffer D
10:	Multi-dimensional arrays (s_i, a_i, r_i, s_{i+1}) randomly sampling the number of batches N from D
11:	$Q_\omega(s_t, a_t) = E_{r_{i\geq t}, s_{i>t} \sim E, a_{i>t} \sim \pi}[R_t \mid s_t, a_t]$ # Online network of Critic network
12:	$Q^{\mu}(s_t, a_t) = E_{r_t, s_{t+1} \sim E}[r(s_t, a_t) + \gamma Q^{\mu}(s_{t+1}, a_{t+1}))]$ # Target network of Critic network
13:	$L(\theta^Q) = E[(Q((s_t, a_t \mid \theta^Q) - r(s_t, a_t) + \gamma Q^{\mu}(s_{t+1}, \mu(s_{t+1}) \mid \theta^Q))^2)]$# Minimize the loss function to update the Critic network
14:	$\nabla_{\theta^{\mu}} J \approx E_{s_t \sim \rho^{\beta}}\left[\nabla_{\theta^{\mu}} Q(s, a \mid \theta^Q)\mid_{s=s_t, a=\mu(s_t \mid \theta^{\mu})}\right]$# The Actor network is updated using policy gradients
15:	$\omega' \leftarrow \varepsilon\omega + (1-\varepsilon)\omega'$# Update the target network parameters
16:	$\theta' = \varepsilon\mu + (1-\varepsilon)\theta'$# Update the target network parameters
17:	Get current status $s_i = \{c_i^{cpu}, c_i^{memory}, c_i^{io}\}$
18:	$a_t = \pi_\theta(s_i) + \varepsilon$; # Execution action
19:	**end for**
20:	**end for**
21:	**Return** a_t; # The resource adjustment plan is displayed

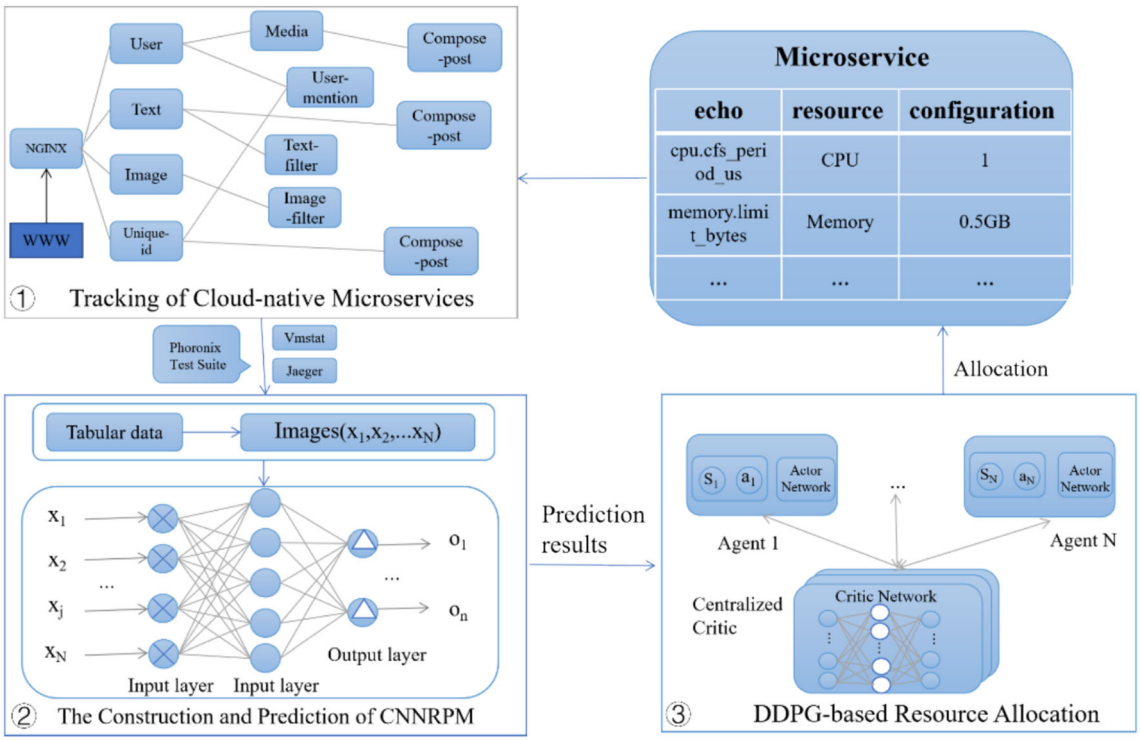


Fig. 5 An example of CNNRPM-based resource allocation

Example Assume that the QoS-specified response time is within the range of 1–5. When the response time is less than 1, the prediction result is 0, indicating that the allocated resources are insufficient to meet the current task's requirements, and more resources need to be allocated. When the response time falls within the range of 1–5, the prediction result is 1, indicating that the allocated resources meet the current task's requirements, and the resource allocation is reasonable. When the response time exceeds 5, the prediction result is 2, indicating that excessive resources are allocated, leading to resource waste, and the resource allocation needs to be adjusted appropriately.

When the response time predicted by CNNRPM exceeds the QoS-specified range, the DDPG strategy is executed to adaptively adjust the resources.

Assume that the resource allocation for this microservice node is $X = \{x_{CPU} = 4, x_{memory} = 512MB, x_{I/O} = 1GB\}$. Based on the physical resources of the local server, the adjustment ranges for each type of resource for the microservice node are determined as follows: $x_{CPU} \in [1,20]$, $x_{memory} \in [512MB, 4096MB]$, $x_{I/O} \in \{1GB, 4GB\}$. Three resource adjustment schemes are proposed as follows:

$$a_1 = \{c_{cpu} = 8, c_{memory} = 1024MB, c_{I/O} = 1GB\}$$
$$a_2 = \{c_{cpu} = 10, c_{memory} = 1536MB, c_{I/O} = 2GB\}$$
$$a_3 = \{c_{cpu} = 12, c_{memory} = 2048MB, c_{I/O} = 3GB\}$$

Next, set $\alpha = 0.5$, $\beta = 0.3$, and $\gamma = 0.2$ in Eq. (3), and use this formula to calculate the reward values r corresponding to the above three resource adjustment schemes. The results are as follows:

When adopting scheme a_1, the values of the elements in s_i are $c_{cpu} = 8, c_{memory} = 1024MB, c_{I/O} = 1GB$, therefore, the reward value is $r_1 = 0.5 * \frac{4}{8} + 0.3 * \frac{512}{1024} + 0.2 * \frac{1}{1} = 0.6$. When adopting scheme a_2, the values of the elements in s_i are $c_{cpu} = 10, c_{memory} = 1536MB, c_{I/O} = 2GB$, therefore, the reward value is $r_2 = 0.5 * \frac{4}{10} + 0.3 * \frac{512}{1536} + 0.2 * \frac{1}{2} = 0.324$. When adopting scheme a_3, the values of the elements in s_i is $c_{cpu} = 12, c_{memory} = 2048MB, c_{I/O} = 3GB$, therefore, the reward value is $r_3 = 0.5 * \frac{4}{12} + 0.3 * \frac{512}{2048} + 0.2 * \frac{1}{3} = 0.336$.

Subsequently, set $\gamma = 0.7$ in Eq. (6) and use this formula to calculate the target values of the value function for the above resource adjustment schemes. The results are as follows:

$$Q(s, a_1) = 0.6 + 0.7 * 0.6 = 1.02$$
$$Q(s, a_2) = 0.324 + 0.7 * 0.3 = 0.534$$
$$Q(s, a_3) = 0.336 + 0.7 * 0.3 = 0.546$$

From the above calculations, it can be concluded that the value function Q is maximized when resource allocation scheme a_1 is adopted. Therefore, scheme 1 is ultimately selected as the resource adjustment strategy.

4 Experiments

We conduct extensive experiments in response to the following research questions (RQ):

RQ1: Compared with other tabular data prediction models, can our proposed CNNRPM predict the response time of microservice nodes more effectively?

RQ2: Can the resource scheduling model based on DDPG maximize the resource utilization compared with the other baselines?

4.1 Experimental setup

Firstly, we applied Docker to deploy several light-weighted containers on two physical hosts, whose details are given in Table 2, to simulate the cloud-native environment. The kernel version for each of the container is Linux 5.10.102.1-microsoft-standard-WSL2, and the operating system is Ubuntu 20.04.5 LTS.

Then, we run Deathstarbench as the real-time applications, and then applied the *Phoronix-Test-Suite* to simulate the resource competition from the perspective of CPU, memory, and I/O. At the same time, the processes management mechanism *Cgroup* are used to alter resource allocation, whose details are given in Table 3, the Linux profiling tool *vmstat* and distributed tracing system *Jaeger* were used to collect data.

4.2 RQ1. Resource pPrediction of CNNRPM

In this part, FIRM, VIME (sl-only, semi-only, self-semi-sl) [30], Scarf [31], Regressor, Decision tree, XGboost, Random Forest and Naive_Bayes are used as baselines. Generally speaking, FIRM is a representative framework for cloud-native resource allocation, VIME and Scarf are two newly proposed deep learning-based models for tabular data prediction, while the others are traditional machine learning method. The details are given as follow:

- FIRM, a representative framework to achieve the fine-grained resource management for microservice, adopts SVM to predict microservice node performance. The parameter is set to learning rate = 0.1, batch_size = 128.
- VIME (Value Imputation and Mask Estimation), a semi-supervised deep learning framework, comes in three versions: sl-only, semi-only, and self-semi-sl. The first one uses multi-layer perceptron (MLP) for training, the second adopts Semi-Supervised Learning for training, and the last one combines self-supervised learning and semi-supervised learning. The parameters are set as maxdepth = 3, learning-rate = 0.1, batch_size = 128.
- Scarf (Scalable and Accurate Recurrent Forest), a scalable and accurate recurrent forest model, captures the regularities in tabular data with the integration of decision trees. The parameters are set as hidden layers = 8, batch_size = 128, corruption_rate = 0.6, learning-rate = 0.1.
- DT (Decision tree), a supervised learning algorithm, uses the idea of classification to build a mathematical

Table 2 The details of simulation environment

	Physical host 1	Physical host 2
CPU type	12th Gen Inte(R) Core(TM) i5-12500H	12th Gen Inte(R) Core(TM) i7-12700H
CPU frequency	3.2 GHz	2.6 GHz
Memory type	16 GB DDR5	DDR4
Memory frequency	6400 MHz	3200 MHz
Hard disk type	512 GB SSD	1024 GB SSD
Last level cache(LLC)	18 MB	24 MB

Table 3 The details of data collection

Feature	Feature value
cpu.shares	512 M, 1024 M, 2048 M, 3072 M
cpu.cfs_period_us	100000us, 150000us, 200000us, 500000us, 1000000us
cpu.cfs_quota_us	− 1us, 100000us, 200000us, 400000us, 500000us
cpuset.cpus	3, 7, 9, 11, 15, 19
memory.limit_in_bytes	512 M, 1024 M, 1536 M, 2048 M, 4096 M
memory.swappiness	10, 20, 30, 40, 50, 60

model according to the characteristics of data, so as to achieve the goal of data screening and decision.

- XGBoostis an efficient gradient boosting decision tree algorithm, and the parameters are set as max-depth = 5, learning_rate = 0.1, estimators = 100.
- RF (A Random Forest), an ensemble learning method that consists of multiple decision trees, whose parameters are set as max-depth = 5, maxfeature = 5, iteration = 100.
- NB (Naive Bayes) is a simple probabilistic classifier based on Bayes' theorem, which assumes that features are independent of each other, and then makes the model easy to implement and understand.

As for the metrics, we adopt Accuracy, Precision, Recall and F1 score to verify the prediction effectiveness based on CNNRPM. The calculation of the above-mentioned metrics are given in Eqs. (1–4).

First, accuracy is an important indicator of the overall performance of the model, which represents the proportion of the number of samples correctly predicted to the total number of samples. The accuracy rate is calculated as follows:

$$f(accuracy) = \frac{N\left(T_{actual} = T_{pre}\right)}{N_{total}} \tag{11}$$

where T_{actual} is the ground truth, T_{pre} is the prediction, $N\left(T_{actual} = T_{pre}\right)$) is the number of correct prediction, and N_{total} indicates the number of total samples.

Next,precision is a measure of the proportion of a model that is actually positive in a sample that is predicted to be positive, which can be calculated by Eq. (12).

$$f(precision) = \frac{TP}{TP + FP} \tag{12}$$

where, TP represents the number of samples that are true positive; FP is the false positive. The higher value of precision is, the better the prediction can be obtained.

The recall rate measures the proportion of all samples that are actually positive be correctly predicted to be positive by the model. The recall rate is calculated as follows:

$$f(recall) = \frac{TP}{TP + FN} \tag{13}$$

where, FN is the false negative, and the higher value of recall is, the better the prediction effect can be got.

Finally, the F1 score is the harmonic average of accuracy and recall, which takes the accuracy and recall into account, and the calculation is given in Eq. 14.

$$F1 = \frac{2 * f(precision) * f(recall)}{f(precision) + f(recall)} \tag{14}$$

To ensure the credibility of the experimental results, we have incorporated t-tests into the evaluation to verify whether the differences observed in the experimental results are statistically significant rather than due to chance. The t-test is a commonly used parametric test method suitable for comparing the mean differences between two sets of data.

The experimental comparisons are given in Table 4, where the bolded font represents the best performing algorithm in the current metrics, and the underlined font indicates the next best method. It can be seen from Table 4 that the CNNRPM has achieved the best performance in all metrics, and its accuracy, recall rate and F1 score are all higher than 99%, even up to 100%, which is significantly higher than other algorithms, and can predict the response time of microservice nodes more accurately, but with the lower standard deviation. FIRM, which makes predictions directly based on data tables, has a slightly lower accuracy, recall rate and F1 score than CNNRPM. XGboost came in second, with both its recall rate and F1 score of 0.98, slightly lower than CNNRPM, but still showing high predictive accuracy. XGboost performs well thanks to its powerful feature processing capabilities and integrated learning framework for capturing complex relationships in data. However, compared with CNNRPM, XGboost falls slightly short in the following aspects: first of all, CNNRPM adopts CNN to extract tabular features in greater depth, and then handle nonlinear relationships better. Secondly, CNNRPM has more balanced indicators, indicating that it has a strong ability to identify positive and negative samples, while XGboost may have a certain deviation between the accuracy rate and the recall rate. In addition, all indicators of CNNRPM algorithm are relatively balanced, and all scores reach 99%, indicating that it has a strong ability to identify positive and negative samples, and can better adapt to different data distributions.

In conclusion, CNNRPM is the most effective model in predicting resource performance among all methods, and its prediction accuracy, stability, reasonable model structure, interpretability and generalization ability are superior to other methods.

4.3 RQ2. Comparative test of resource allocation model based on DDPG

We now evaluate the ability of DDPG-based resource adjustment to meet QoS while reducing resource consumption.

The parameter settings for training DDPG using feature-performance data are as follows: For the reward function shown in Eq. (3), the parameters are set as $\alpha = 0.5$, $\beta = 0.3$, and $\gamma = 0.2$, representing the weights of CPU, memory, and I/O resources in the resource scheduling process respectively. Other key parameters include a minibatch size of 64, a replay buffer capacity of 100, a

Table 4 The comparisons of different metrics with 95% confidence intervals

	Accuracy	Precision	Recall	F1 score
Scarf	0.83 ± 2.32	0.84 ± 7.48	0.99 ± 6.11	0.91 ± 5.22
VIME (sl-only)	0.64 ± 3.22	0.73 ± 4.56	0.64 ± 4.23	0.67 ± 4.61
VIME (semi-only)	0.28 ± 1.79	0.73 ± 2.23	0.28 ± 4.22	0.36 ± 3.27
VIME (self-semi-sl)	0.57 ± 1.78	0.71 ± 3.46	0.57 ± 3.56	0.62 ± 3.56
FIRM	0.96 ± 2.99	0.95 ± 2.56	0.96 ± 2.89	0.95 ± 3.52
Regressor	0.94 ± 2.74	0.09 ± 2.68	0.01 ± 3.45	0.05 ± 2.74
DecisionTree	0.98 ± 2.78	0.96 ± 6.45	0.96 ± 4.12	0.96 ± 2.75
XGboost	0.99 ± 2.25	0.99 ± 2.23	0.98 ± 5.31	0.98 ± 2.71
Random_forest	0.98 ± 2.26	0.98 ± 2.75	0.93 ± 4.12	0.96 ± 2.14
Naïve_bayes	0.47 ± 1.79	0.47 ± 2.25	0.62 ± 3.21	0.55 ± 1.12
CNNRPM	**1.00 ± 1.76**	**0.99 ± 1.74**	**0.99 ± 2.17**	**0.99 ± 1.12**

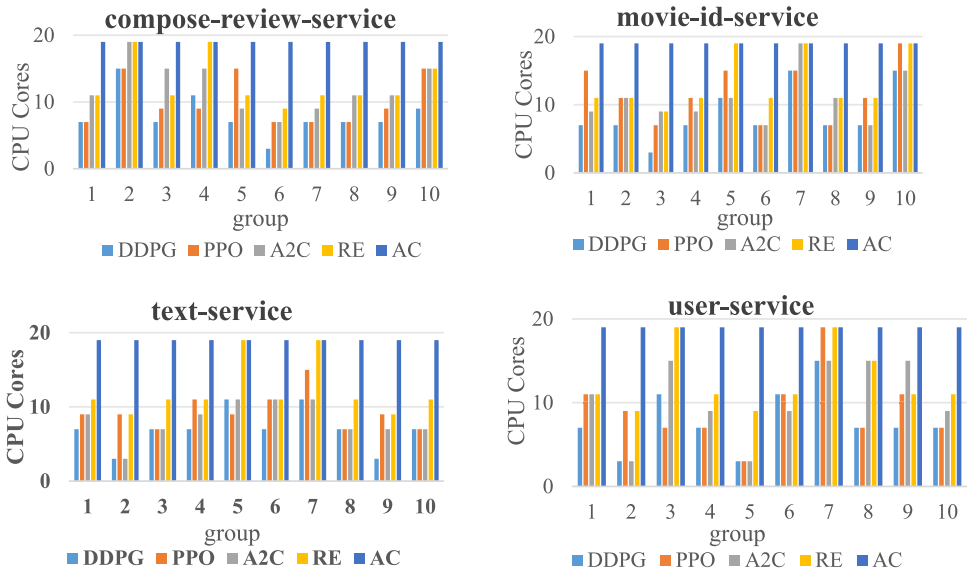


Fig. 6 Number of CPU cores allocated by different schemes

learning rate of 0.003 for the Actor network, a learning rate of 0.03 for the Critic network, and a discount factor for the reward value in each action step. The DDPG code is implemented in Python 3.0 and deployed on the PyTorch framework. Both the Actor and Critic networks consist of 4 fully connected layers, each containing 40 neurons. The activation function between layers is ReLU(), while the final layer uses Tanh() as the activation function.

We compared DDPG with PPO[32], A2C, auto scaling and reservation, the first two are another representative deep reinforcement methods.

- PPO (Proximal Policy Optimization) is a reinforcement learning algorithm based on policy gradients. It updates the strategy through near-end strategy optimization to achieve stable and efficient training results. Set the parameters to alpha = 0.5, episodes = 64, verbose = 1, and timesteps = 4000.
- A2C (Advantage Actor-Critic) algorithm is a deep reinforcement learning algorithm based on policy gradient, which uses the Actor-Critic framework. In this framework, the Actor is responsible for learning a policy, that is, the probability distribution of selected actions in a given state; The Critic is responsible for learning a value function, that is, to estimate the value of a state or state-action pair. Set the parameters to alpha = 0, episodes = 64, verbose = 1, and timesteps = 4000.
- Autoscaler (AC) refers to the dynamic allocation of computing resources according to the current demand or load of the system. Other metrics that provide microservices for algorithm scaling decisions include cloud monitoring metrics and CPU and memory

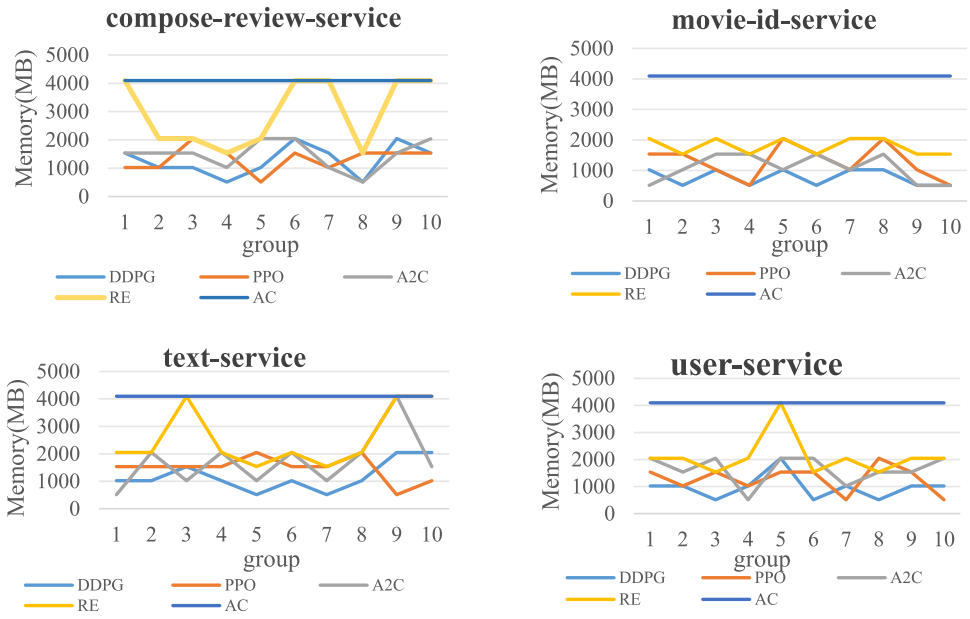


Fig. 7 Memory allocated by different schemes

utilization. By analyzing these metrics, the algorithm scaling algorithm can make informed decisions about expanding or reducing resources.

- The resource reservation (RE) ensures the quality of critical services by reserving some resources for high-priority needs. The algorithm evaluates the current resource situation and the forecast of future demand, dynamically adjusts the reservation strategy, and aims to achieve efficient and fair resource allocation.

In order to understand the advantages of DDPG in improving resource utilization, ten groups of data were selected according to different amounts of CPU, memory and I/O resources of microservice nodes, and DDPG, PPO, A2C, automatic scaling and reservation algorithms were used to allocate CPU cores and memory for each microservice, as shown in Figs. 6 and 7.

As illustrated in the Fig. 6, AC shows great resource wastage compared to DDPG, allocating roughly twice as many CPU cores on average. This is mainly due to its failure to adapt to the dynamic resource demands of microservice nodes. RE can allocate CPU cores and memory flexibly based on changing requirements, but it still results in noticeable waste compared to DDPG. The CPU utilization of the DDPG algorithm is 38.32% higher than that of RE, primarily because it allocates resources based on the last peak demand.

Experimental results indicate that both the PPO and A2C algorithms outperform AC and RE in CPU allocation but are still slightly less effective than the DDPG algorithm. PPO constrains policy updates by clipping the probability ratio to avoid instability from excessive updates, combining strategy gradient methods with value function estimation to improve policy performance through multiple iterations and simplified objectives. PPO effectively balances exploration and exploitation in resource allocation but may not explore sufficiently in high-dimensional continuous action spaces, and its use of historical data is not as efficient as DDPG. Statistically, DDPG saves 22.8% of CPU resources compared to PPO.

The A2C enhances data collection efficiency by gathering experience in parallel with multiple threads and uses the advantage function for policy updates to reduce variance. While A2C excels in handling massively parallel environments, it is primarily suited for discrete action spaces, limiting its application to continuous action space resource allocation. In contrast, the DDPG, with its deterministic strategy and offline learning characteristics, is more suitable for high-dimensional continuous action space resource allocation, with a CPU utilization 24.52% higher than A2C.

Figure 7 reveals a detailed analysis of the memory allocation by DDPG versus the baselines throughout the entire testing. As depicted in Fig. 6, on average, DDPG outperforms the PPO, A2C, AC, and RE by conserving 21.91%, 29.27%, 55.43%, and 74.39% of memory resources, respectively, with a peak saving of up to 80.82%.

In summary, in comparison to PPO, A2C, algorithm scaling, and reservation algorithms, DDPG demonstrates superior capability in effectively allocating CPU and memory demonstrates superior capability in effectively allocating CPU and memory resources.

Furthermore, to verify if DDPG can improve resource utilization while ensuring QoS, we further evaluated the response time of microservice, and give details in Fig. 8.

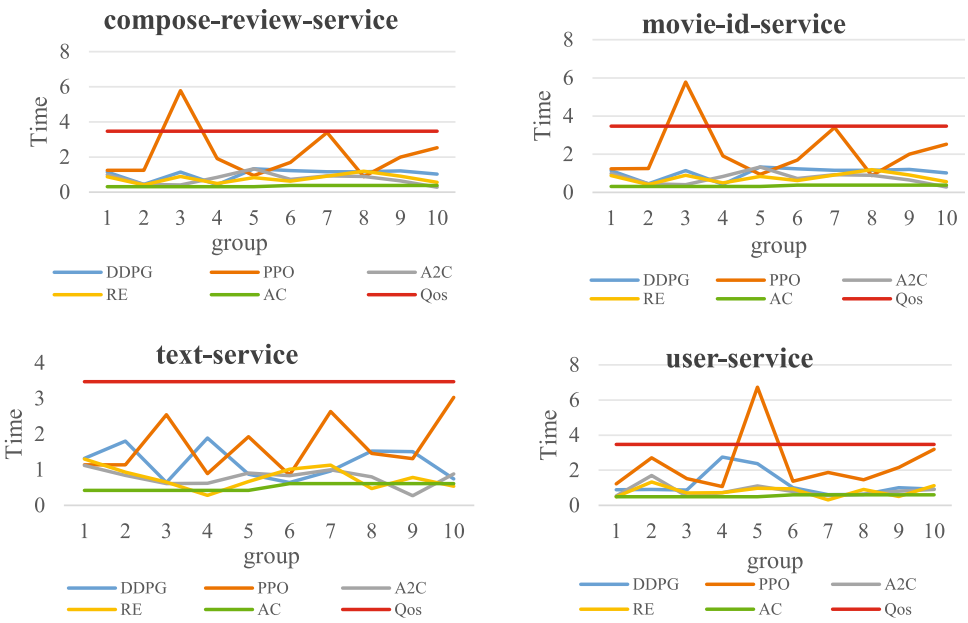


Fig. 8 The average response time of different schemes

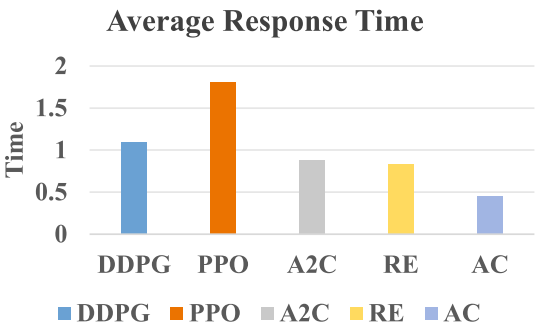


Fig. 9 The average response time of different schemes

Figure 8 shows the response time of microservice nodes in different schemes, and Fig. 9 shows the average response time of microservice nodes. When the response time of microservice nodes is lower than 1 ms, it proves that the allocated microservice resources are too many and there is a waste of resources. When the response time of the microservice node is between 1 and 5 ms, it proves that the resources allocated can just meet the resource requirements of the current microservice node. When the response time is greater than 5 ms, it proves that the resources allocated to the microservice node are insufficient to meet the resource requirements of the current node, and more resources need to be allocated to it.

From the experimental results we can see that DDPG is not always optimal in terms of response time and the average response time is second only to PPO. This is due to the fact that DDPG employs actor-critic, a dual-network architecture, which is higher in complexity than any other methods. However, from the experimental results presented in Figs. 6 and 7, we can see that DDPG is able to effectively conserve the server's CPU and memory resources. Therefore, in real situations, users should weigh the relationship between the allocation efficiency of resources and the time efficiency of the allocation algorithm to make the optimal choice.

5 Conclusion and discussion

In this paper, we propose a CNN-based adaptive resource allocation framework for cloud-native architectures that dynamically adjusts system resources including CPU and memory of microservice nodes. Extensive experimental results demonstrate that our proposed method reduce CPU consumption by 22.8% and memory usage by 21.91% compared to state-of-the-art baselines.

While CPRAM demonstrates significant improvements in resource allocation and QoS guarantees for cloud-native microservices, it is not without limitations. Firstly, the performance of CPRAM relies heavily on the accuracy of the CNN-based prediction model. Although CNNs are robust to sparse and incomplete data, their effectiveness may degrade in highly dynamic environments where resource usage patterns change abruptly or unpredictably. Secondly, the DDPG-based resource allocation mechanism, while effective, involves a relatively high computational cost during the training phase. This could limit its applicability in real-time scenarios where rapid decision-making is critical. Although we have optimized the training process by using techniques such as experience replay and minibatch updates, further improvements are needed to reduce the training time and computational complexity. Thirdly, CPRAM currently focuses solely on optimizing

latency as the primary QoS metric. While latency is a critical factor in many cloud-native applications, other metrics such as throughput, energy consumption, and cost efficiency are also important in practical scenarios.

For the future, we plan to explore multi-agent collaborative learning to investigate how local decisions from individual agents can be efficiently integrated to optimize global resource allocation. Additionally, we will incorporate online learning techniques to dynamically update the parameters of the reinforcement learning algorithm, enabling the framework to adapt to evolving software and hardware environments in real time.

Author contributions Yu Peng was responsible for proposing the proposal, performing the experimental validation, and the writing part of the paper. Jia Hao was responsible for proofreading. Yang Chen was responsible for writing the experimental part, and Jianhou Gan is responsible for reviewing the full paper.

Funding This work is supported by Yunnan International Joint R&D Center of China-Laos-Thailand Educational Digitalization (202203AP140006), Youth Project of Applied Basic Research Program of Yunnan Province (No. 202201AU070050), Key Project of Applied Basic Research Program of Yunnan Province (No. 202201AS070021), Talent Development Program for Yunnan-Youth Talent Project.

Data availability No datasets were generated or analysed during the current study.

Declarations

Conflict of interest The authors declare no competing interests.

Ethical approval Not applicable.

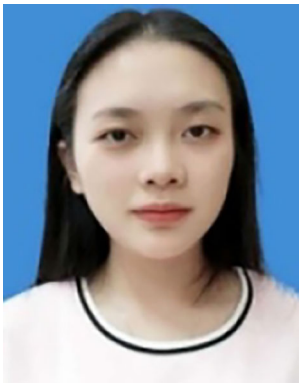
References

1. Kratzke, N., Quint, P.-C.: Understanding cloud-native applications after 10 years of cloud computing—a systematic mapping study. J. Syst. Softw. **126**, 1–16 (2017)
2. Sotiriadis Nathan Cruz Coulson Stelios and Bessis Nik: Adaptive microservice scaling for elastic applications. IEEE Internet Things J. **7**(5), 4195–4202 (2020)
3. Fu, K., Zhang, W., Chen, Q., et al.: Adaptive resource efficient microservice deployment in cloud-edge continuum. IEEE Trans. Parallel Distrib. Syst. **33**(8), 1825–1840 (2022)
4. Chouliaras, S., Sotiriadis, S.: Real-time anomaly detection of NoSQL systems based on resource usage monitoring. IEEE Trans. Industr. Inf. **16**(9), 6042–6049 (2020)
5. Bhardwaj, R., Kandasamy, K., Biswal, A., et al.: Cilantro: performance-aware resource allocation for general objectives via online feedback. In: USENIX Symposium on Operating Systems Design and Implementation. (2023)
6. Qiu, H., Banerjee, S.S., Jha, S., et al.: FIRM: an intelligent fine-grained resource management framework for SLO-oriented microservices. In: Proceedings of the 14th USENIX Conference on Operating Systems Design and Implementation. 2020, USENIX Association. p. Article 46.
7. Guo, Y., Liu, F., Xiao, N., et al.: PARA: Performability-aware resource allocation on the edges for cloud-native services. Int. J. Intell. Syst. **37**(11), 8523–8547 (2022)
8. Moghaddam, S.K., Buyya, R., Ramamohanarao, K.: Performance-aware management of cloud resources: a taxonomy and future directions. ACM Comput. Surv. **52**(4), Article 84 (2019)
9. Liao, T.-L., Peng, C.-Y., Hou, Y.-Y.: Application of multi-party computation and error correction with image enhancement and convolution neural networks based on cloud computing. IET Image Proc. **17**(6), 1931–1950 (2023)
10. Yang, J., Wang, Y., Li, Z.: Inverse order based optimization method for task offloading and resource allocation in mobile edge computing. Appl. Soft Comput. **116**, 108361 (2022)
11. Gan, Y., Zhang, Y., Cheng, D., et al.: An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems. In: Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems. pp. 3–18 (2019)
12. Rzadca, K., Findeisen, P., Swiderski, J., et al.: Autopilot: workload autoscaling at Google. In: Proceedings of the Fifteenth European Conference on Computer Systems. pp. 1–16 (2020)
13. Zhang, Y., Hua, W., Zhou, Z., et al.: Sinan: ML-based and QoS-aware resource management for cloud microservices. In: Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems. pp. 167–181 (2021)
14. Crankshaw, D., Wang, X., Zhou, G., et al.: Clipper a low-latency online prediction serving system. Comput. Sci. (2017)
15. Joseph, C.T., Chandrasekaran, K.: Nature-inspired resource management and dynamic rescheduling of microservices in Cloud datacenters. Concurr. Comput. Pract. Exp. (2021). https://doi.org/10.1002/cpe.6290
16. Erdei, R., Toka, L.: Minimizing resource allocation for cloud-native microservices. J. Netw. Syst. Manage. **31**(2), 35 (2023)
17. Khan, T.M.G., Taheri, J., Al-Dulaimy, K.A.: PerfSim: a performance simulator for cloud native microservice chains. IEEE Trans. Cloud Comput. **11**(2), 1395–1413 (2023)
18. Byeonghui, J., Jueun, J., Young-Sik, J.: Proactive resource autoscaling scheme based on SCINet for high-performance cloud computing. IEEE Trans. Cloud Comput. **11**(4), 3497–3509 (2023)
19. Masouros, D., Xydis, S., Soudris, D.: Rusty: runtime interference-aware predictive monitoring for modern multi-tenant systems. IEEE Trans. Parallel Distrib. Syst. **32**(1), 184–198 (2021)
20. Zhang, Y., Zhou, Z., Elnikety, S., Delimitrou, C.: Analytically-driven resource management for cloud-native microservices. Comput. Sci. (2024)
21. Wang, Z., Li, P., Liang, C.-J.M., et al.: Autothrottle: a practical bi-level approach to resource management for SLO-targeted microservices. Comput. Sci. (2022)
22. Shutian, L., Chenyu, L., Kejiang, Ye., et al.: Optimizing resource management for shared microservices: a scalable system design. ACM Trans. Comput. Syst. **42**(1–2), 1–28 (2024)
23. Chen, S., Delimitrou, C., and Martínez, J.F.: PARTIES: QoS-aware resource partitioning for multiple interactive services. In: Proceedings of the twenty-fourth international conference on architectural support for programming languages and operating systems. Association for Computing Machinery: Providence, RI, USA. pp. 107–120 (2019)
24. Guo, H., Cao, H., He, J., et al.: POBO: Safe and optimal resource management for cloud microservices. Perform. Evaluat. **162**, 102376 (2023)

25. Kim, A., Lee, J., Kim, M.: Resource management model based on cloud computing environment. Int. J. Distrib. Sensor Netw. **12**(11), 550147716676554 (2016)
26. Baarzi, A.F. and Kesidis, G.: SHOWAR: right-sizing and efficient scheduling of microservices. In: Proceedings of the ACM Symposium on Cloud Computing. Association for Computing Machinery: Seattle, WA, USA. pp. 427–441 (2021)
27. Nawrocki, P., Grzywacz, M., Sniezynski, B.: Adaptive resource planning for cloud-based services using machine learning. J. Parallel Distrib. Comput. **152**, 88–97 (2021)
28. Panwar, R., Supriya, M.: Dynamic resource provisioning for service-based cloud applications: A Bayesian learning approach. J. Parallel Distrib. Comput. **168**, 90–107 (2022)
29. Kumar, R.A., Kartheeban, K.: Resource allocation using dynamic pricing auction mechanism for supporting emergency demands in cloud computing. J. Parallel Distrib. Comput. **158**, 213–226 (2021)
30. Yoon, J., Zhang, Y., Jordon, J., et al.: VIME: extending the success of self- and semi-supervised learning to tabular domain. In: Neural Information Processing Systems (2020)
31. Bahri, D., Jiang, H., Tay, Y., et al.: SCARF: Self-Supervised Contrastive Learning using Random Feature Corruption. CoRR, (2021). arxiv:2106.15147
32. Schulman, J., Wolski, F., Dhariwal, P., et al., Proximal Policy Optimization Algorithms. ArXiv, (2017). arxiv:1707.06347.
33. Aslani, A., Ghobaei-Arani, M.: Machine learning inference serving models in serverless computing: a survey. Computing **107**, 47 (2025). https://doi.org/10.1007/s00607-024-01377-9
34. Ghorbian, M., Ghobaei-Arani, M., Esmaeili, L.: A survey on the scheduling mechanisms in serverless computing: a taxonomy, challenges, and trends. Clust. Comput. **27**, 5571–5610 (2024). https://doi.org/10.1007/s10586-023-04264-8
35. Ghorbian, M., Ghobaei-Arani, M.: Function offloading approaches in serverless computing: a survey. Comput. Electric. Eng. **120**(1), 109832 (2024)
36. Anisha kumari, Bibhudatta Sahoo, and Ranjan Kumar Behera. Mitigating cold-start delay using warm-start containers in serverless platform. In 2022 IEEE 19th India Council International Conference (INDICON), pp. 1–6. IEEE, 2022.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.



Yu Peng born in 2000, received her B.S degree from the Guiz hou Minzu University. Cur rently, she is a master student in the Information School of Science in Yunnan Normal University. Her research interest covers cloud computing and edge computing.



Jia Hao received the MS.D. and Ph.D. in computer science from Wuhan University of Technology and Yunnan University in 2015 and 2020 respectively. She is currently a lecturer and a postdoctoral research fellow in the Minister of Education at Yunnan Normal University, Kunming, China. Her research interests include massive data analysis, uncertainty in artificial intelligence, cloud computing, and edge computing.



Yang Chen born in 1997, received his B.S degree from the Southwest Petroleum University. Currently, he is a master student in the Information School of Science in Yunnan Normal University. His research interest covers cloud computing and edge computing.